

Design Document (DD)

An Energy-Efficient Dual Hydroponic System for Organic/Inorganic Capsicum Chili Farming

Project ID: 25-26J-035

**Individual component: Automated climate zone that
adapts to weather and predicts fungal risks for
healthy capsicum growth**

**Kariyawasam K.S.T.S.A
IT22617996**

**BSc (Hons) Computer Systems & Network
Engineering**

Supervisor: Ms. Shashika Lokuliyana

1/11/2026

TABLE OF CONTENTS

1. Introduction	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Definitions, Acronyms, and Abbreviations	5
1.4 Overview	6
2. Overall Description.....	7
2.1 Product Perspective.....	7
2.1.1 System Interfaces	8
2.1.2 User Interfaces	8
2.1.3 Hardware Interfaces	8
2.1.4 Software Interfaces	8
2.1.5 Communication Interfaces	9
2.1.6 Memory Constraints.....	9
2.1.7 Operations	9
1. Sensor data collection (temp/humidity).....	9
2. Automatic fan/mist activation based on on/off control.	9
3. Leaf image upload and disease prediction via mobilenet cnn.	9
4. Backend api handles communication with mobile app and esp32.	9
2.1.8 Site Adaptation Requirements.....	9
2.2 Product Functions	9
2.3 User Characteristics	10
2.4 Constraints	10
2.5 Assumptions and Dependencies	10
2.6 Apportioning of Requirements	10
3. Specific requirements	12
3.1 External interface requirements	12
3.1.1 User Interfaces	12
3.1.2 Hardware Interfaces	12
3.1.3 Software Interfaces	13
3.1.4 Communication Interfaces	13
3.2 Architectural Design	14
3.2.1 High-Level Architectural Design.....	14

3.2.2	Hardware and Software Requirements with Justification.....	15
3.2.3	Risk Mitigation Plan with Alternative Solutions	15
3.2.4	Cost-Benefit Analysis for the Proposed Solution	15
3.3	Performance Requirements	16
3.4	Design Constraints	16
3.5	Software System Attributes.....	17
3.5.1	Reliability.....	17
3.5.2	Availability.....	17
3.5.3	Security	17
3.5.4	Maintainability.....	17
3.5.5	Other Requirements	17
4.	Supporting information.....	18
4.1	Appendices.....	18
5.	References	21

1. INTRODUCTION

1.1 Purpose

The purpose of this document is to provide a comprehensive design framework for the proposed IoT-enabled dual hydroponic climate control system. This system integrates sensor-based monitoring, automated climate control, and machine learning-based leaf disease detection for capsicum chili cultivation.

Objective: Describe all design decisions, interfaces, operational flow, hardware/software components, and implementation strategies used in the PP1 phase.

Significance: Manual greenhouse management can lead to inconsistent conditions and delayed disease detection. By combining IoT, ML, and automated control, this system ensures optimal plant growth, early disease identification, and energy-efficient operation.

1.2 Scope

This Design Document (DD) covers the design, implementation, and integration of the following components in the automated dual hydroponic system for capsicum chili cultivation:

1. **ESP32 Microcontroller Integration:**

- Connects and interfaces with temperature and humidity sensors (DHT22).
- Collects real-time environmental data for climate monitoring.

2. **Automatic Fan and Misting Control:**

- Implements basic ON/OFF control to regulate greenhouse temperature and humidity.
- Ensures environmental conditions remain within optimal ranges for capsicum growth.

3. **Machine Learning-Based Leaf Disease Detection:**

- Utilizes a pre-trained MobileNet Convolutional Neural Network (CNN) model.
- Allows users to upload leaf images via a mobile app for automated disease identification.

4. **Backend Development Using Flask:**

- Handles API requests from the mobile app and ESP32.

- Performs ML inference for disease prediction.
- Stores sensor data and uploaded images for future analysis and reporting.

5. Mobile Application Interface:

- Provides a user-friendly interface for image upload and disease prediction results.
- Displays the predicted disease along with confidence scores to guide user decisions.

This scope defines the PP1 implementation boundaries, focusing on basic automation and ML integration, with future phases planned to include predictive fungal risk control, dew point calculation, and full climate automation

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
IoT	Internet of Things
ESP32	Microcontroller used for sensor interfacing
DHT22	Temperature and humidity sensor
ML	Machine Learning
CNN	Convolutional Neural Network
Flask	Python web framework
API	Application Programming Interface
PP1	Project Proposal Phase 1
ML Model	Pre-trained MobileNet-based CNN

1.4 Overview

The system continuously monitors greenhouse temperature and humidity, controlling fans and misting automatically using ON/OFF logic. A mobile app allows leaf image uploads, which are analyzed using a MobileNet CNN to detect diseases.

Objective: Provide early disease detection and maintain optimal environmental conditions for energy-efficient, high-yield capsicum farming.

2. OVERALL DESCRIPTION

The proposed system is an IoT-enabled automated hydroponic climate control system designed for dual organic and inorganic capsicum chili cultivation. It integrates ESP32 microcontrollers with temperature and humidity sensors (DHT22) to continuously monitor the greenhouse environment. Based on the sensor readings, the system controls the fan and misting systems using simple ON/OFF logic to maintain optimal climate conditions for plant growth. In addition, the system incorporates machine learning capabilities through a pre-trained MobileNet Convolutional Neural Network (CNN) to detect leaf diseases from images uploaded via a mobile application. The Flask-based backend handles all API requests, processes sensor and image data, performs ML inference, and sends results back to the mobile app. This architecture allows farmers, researchers, and hobbyists to monitor crop health, receive disease predictions with confidence scores, and ensure stable environmental conditions, all with minimal manual intervention. The modular design ensures that future enhancements, such as leaf wetness sensors, predictive fungal risk control, and proportional fan/mist adjustments, can be integrated without disrupting the current system. Overall, the system provides a data-driven, energy-efficient, and user-friendly solution for modern precision agriculture.

2.1 Product Perspective

Traditional hydroponic setups rely on manual monitoring and control, which can result in inconsistent conditions and delayed disease detection.

This system combines:

- IoT sensors for real-time climate monitoring
- Automated climate control with fuzzy logic
- ML-based disease detection for proactive intervention

This approach improves crop yield, reduces human effort, and provides data-driven insights for precision agriculture.

2.1.1 System Interfaces

Component	Interface
OS	Windows 10/11 64-bit for backend development
Microcontroller	ESP32 firmware for sensor integration
Mobile App	React Native / Expo for user interaction

2.1.2 User Interfaces

- Mobile app allows image upload, displays disease name and confidence score, and will eventually provide predictive climate control suggestions.
- Backend responds to API requests in JSON format for communication with the app.

2.1.3 Hardware Interfaces

Hardware	Interface	Purpose
DHT22	GPIO of ESP32	Temperature & humidity sensing
Relay Module	GPIO of ESP32	Controls fan and misting system (ON/OFF)
ESP32	Wi-Fi	Sends sensor data to backend

2.1.4 Software Interfaces

Software	Purpose
Python 3.10.5	Backend & ML inference
Flask	Handles API requests from app and ESP32
TensorFlow/Keras	MobileNet CNN for leaf disease detection
React Native / Expo	Mobile app development

2.1.5 Communication Interfaces

- ESP32 communicates with backend over Wi-Fi
- Mobile app communicates with backend using HTTP REST APIs
- Data is exchanged in JSON format

2.1.6 Memory Constraints

- ESP32 internal memory for sensor logic
- Backend server storage for:
 - ML model (leaf.h5)
 - Uploaded images

2.1.7 Operations

1. Sensor data collection (temp/humidity).
2. Automatic fan/mist activation based on on/off control.
3. Leaf image upload and disease prediction via mobilenet cnn.
4. Backend api handles communication with mobile app and esp32.

2.1.8 Site Adaptation Requirements

- Wi-Fi connection required for backend communication.
- Suitable for indoor greenhouse or hydroponic setups.

2.2 Product Functions

1. Automated environmental data collection (temp/humidity).
2. ON/OFF climate control for fan and misting.
3. Leaf disease detection via mobile app image upload.
4. API-based communication between app, ESP32, and backend

2.3 User Characteristics

- Farmers with basic smartphone usage
- Agriculture researchers students
- Hydroponic hobbyists

2.4 Constraints

- **Fan Control:** Only ON/OFF functionality is currently available; speed or proportional control will be implemented in future phases
- **Leaf Wetness Sensor:** Sensor is not yet integrated, so fungal risk prediction is estimated from temperature and humidity
- **Mobile App:** Partially functional; full predictive control integration is pending

2.5 Assumptions and Dependencies

- Stable internet connection for API data
- Proper sensor calibration to ensure accurate readings
- Availability of pre-trained ML model for leaf disease detection

2.6 Apportioning of Requirements

Phase	Implemented / Planned Tasks	Notes
PP1 (Current)	- Temperature & humidity sensors connected to ESP32 - Basic fan & misting ON/OFF control using fuzzy logic - External weather data fetching via API - Image upload and leaf disease detection using ML (MobileNet model)	Leaf wetness sensor not yet connected; fan speed control not available; predictive fungal risk not fully implemented

PP2 (Next Phase)	- Dew point and leaf wetness estimation for fungal risk detection - Optimization of fan & misting based on risk levels - Full climate automation with predictive fungal risk control	Integration with predictive control and app alerts
PP3 (Final Phase)	- Early alerts for disease & climate threats - Daily logs and dashboard visualization	Complete system integration with app and cloud reporting

3. SPECIFIC REQUIREMENTS

3.1 External interface requirements

3.1.1 User Interfaces

- **Mobile app interface for image upload:**

The mobile app allows the user (farmer/researcher) to take or select a leaf image and upload it to the backend for disease detection. This interface must be simple and intuitive, requiring minimal technical knowledge.

- **Display disease name and confidence:**

After processing the image, the backend returns the predicted disease along with a confidence score (percentage). This helps the user understand the likelihood of a specific disease.

This interface bridges the human user with the system's ML capability.

3.1.2 Hardware Interfaces

- **DHT22 → ESP32 (GPIO):**

The DHT22 sensor measures temperature and humidity. Its data is sent to the ESP32 microcontroller through a GPIO pin. This interface is critical because accurate climate data is the basis for automated fan/misting control.

- **Relay → Fan and misting system:**

The ESP32 controls the relay module, which acts as a switch to turn the fan and misting system ON or OFF. This interface ensures safe and reliable electrical control.

These hardware connections form the physical bridge between sensors, actuators, and the control logic running on the ESP32.

3.1.3 Software Interfaces

- **Python backend (Flask):**

The Flask server handles all incoming requests from the app and ESP32. For example, it processes uploaded images and returns ML predictions.

- **ML inference (TensorFlow + Keras):**

The pre-trained MobileNet CNN model runs on the backend to detect diseases in the leaf images.

- **Mobile app via Expo:**

The app communicates with the backend APIs to upload images, fetch predictions, and display results.

These software interfaces ensure data flows smoothly from sensors and user inputs to processing and feedback. They also enable modular updates (e.g., model upgrades or API changes) without touching hardware.

3.1.4 Communication Interfaces

- **HTTP POST for image upload:**

The app sends leaf images to the backend using HTTP POST requests. The backend processes the image and returns results.

- **REST APIs for weather data:**

The backend fetches external weather data (temperature, humidity) to adjust greenhouse conditions.

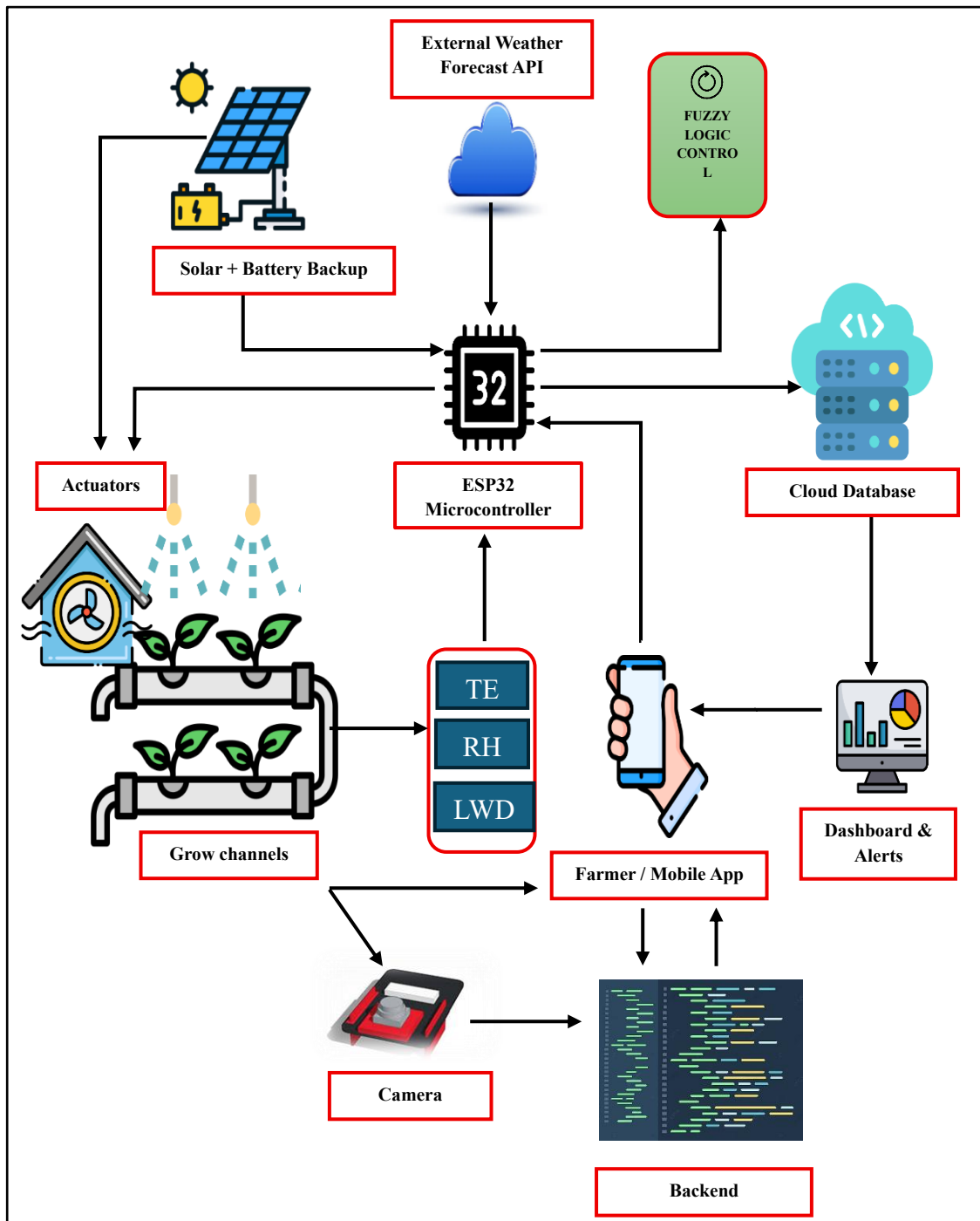
- **Weather API fetch:**

This interface allows the system to compare indoor vs outdoor conditions for predictive climate control.

These communication protocols enable **real-time interaction** between the user, sensors, backend, and external data sources. Without them, neither automated control nor ML-based disease detection would function properly.

3.2 Architectural Design

3.2.1 High-Level Architectural Design



3.2.2 Hardware and Software Requirements with Justification

Component	Version	Purpose
Windows 10/11	64-bit	Backend dev
Python	3.10.5	Backend & ML
Flask	Latest	API handling
TensorFlow/Keras	Latest	ML inference
ESP32	–	Microcontroller
DHT22	–	Temp/humidity sensing
Relay + Fan/Mist	–	Climate control
Expo / React Native	Latest	Mobile app dev

3.2.3 Risk Mitigation Plan with Alternative Solutions

Risk	Mitigation
Sensor failure	Manual override
API failure	Local fallback logic
ML misclassification	Confidence threshold

3.2.4 Cost-Benefit Analysis for the Proposed Solution

The proposed system uses low-cost sensors such as DHT22 and ESP32, which are inexpensive yet reliable for greenhouse monitoring. Open-source software tools like Python, Flask, TensorFlow, and Keras are used to implement the backend, machine learning model, and APIs without additional licensing costs.

Benefits:

- Reduces overall hardware and software expenses compared to commercial automation solutions.
- Improves efficiency by automating climate control and disease detection, saving time and labor.

- Modular design allows the system to expand or upgrade in future phases with minimal additional cost.

3.3 Performance Requirements

The system performance is measured in terms of responsiveness and accuracy. Key requirements include:

- **Sensor reading frequency:** Sensor data (temperature & humidity) is collected every few seconds to ensure timely detection of changes in the greenhouse environment.
- **ML prediction speed:** When a leaf image is uploaded, the MobileNet-based CNN model processes the image and returns a prediction within 2–3 seconds, enabling near real-time disease detection.
- **Fan/mist control:** The system activates fans and misting in real-time based on fuzzy logic rules, ensuring that the climate is maintained efficiently.

3.4 Design Constraints

- **Hardware:** Only temperature & humidity sensors connected; leaf wetness and CO₂ sensors not yet integrated. Fans/misting work in **ON/OFF mode** only.
- **Software:** Fuzzy logic controls basic climate; predictive fungal risk and alerts not yet implemented. ML disease detection uses pre-trained model.
- **Environment:** Requires internet for weather API; designed for Sri Lankan capsicum growth. Solar power may fluctuate.
- **App/User:** Mobile app under development; alerts and dashboard dependent on backend server.
- **Project Scope:** PP1 focuses on basic sensor integration, control, and ML-based image detection; full automation and logging planned for later phases.

3.5 Software System Attributes

3.5.1 Reliability

The system is designed for continuous operation, with retry logic built into the ESP32 and backend to handle sensor communication failures. Reliable ML inference ensures accurate disease prediction.

3.5.2 Availability

The system is intended to operate 24/7, provided network connectivity is available. The backend server handles simultaneous requests from the mobile app and sensors efficiently.

3.5.3 Security

- **CORS control:** Ensures secure API access from authorized apps only.
- **API-based communication:** Data between the mobile app, backend, and ESP32 is transmitted using secure HTTP/REST APIs, preventing unauthorized access.

3.5.4 Maintainability

The system is designed with modularity in mind. Components such as sensors, ML models, or app features can be upgraded independently without affecting other parts of the system.

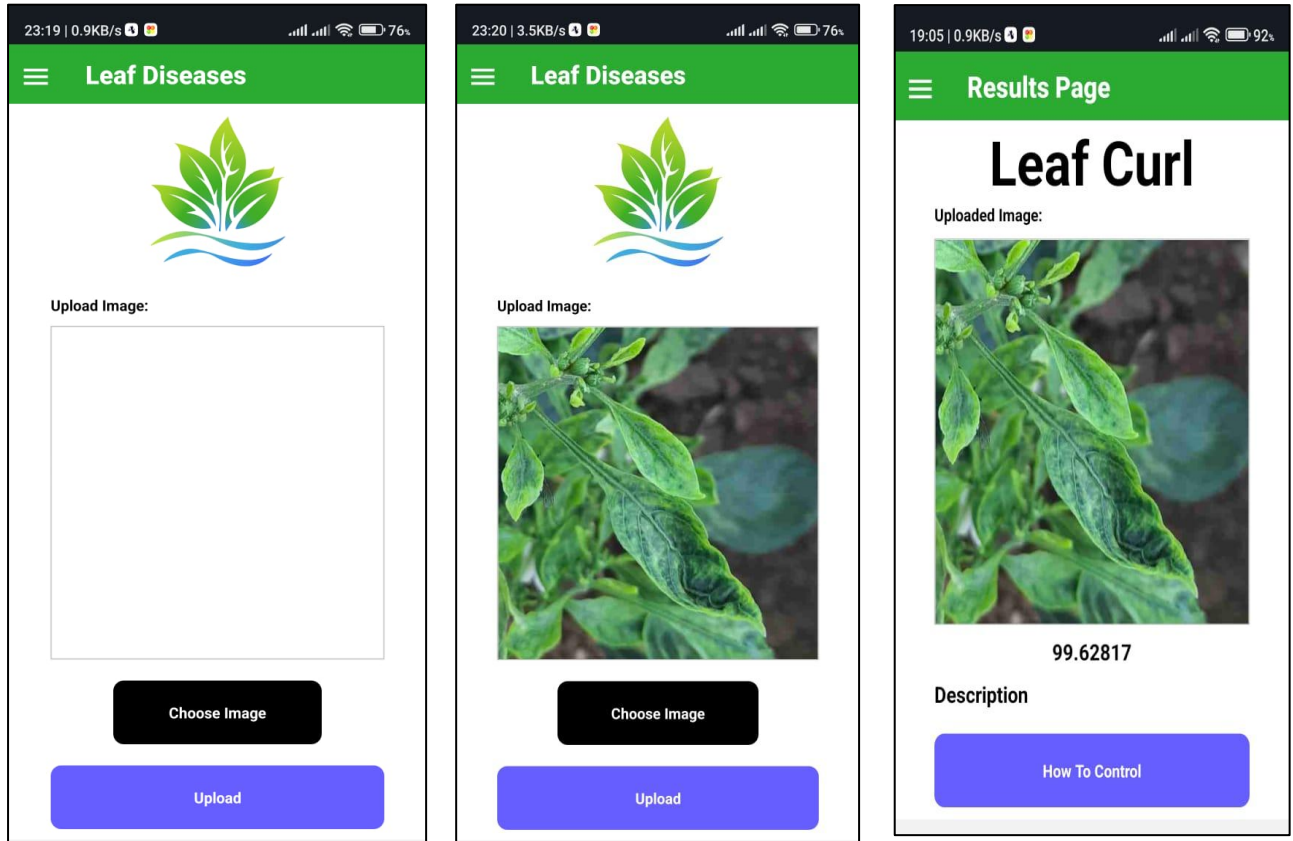
3.5.5 Other Requirements

- **Image dataset expansion planned:** To improve ML model accuracy, more leaf images representing different diseases and healthy leaves will be collected and added.
- **Leaf wetness estimation planned:** Once a leaf wetness sensor is connected, the system will include dew point and leaf wetness-based predictive fungal risk detection for more precise climate control.

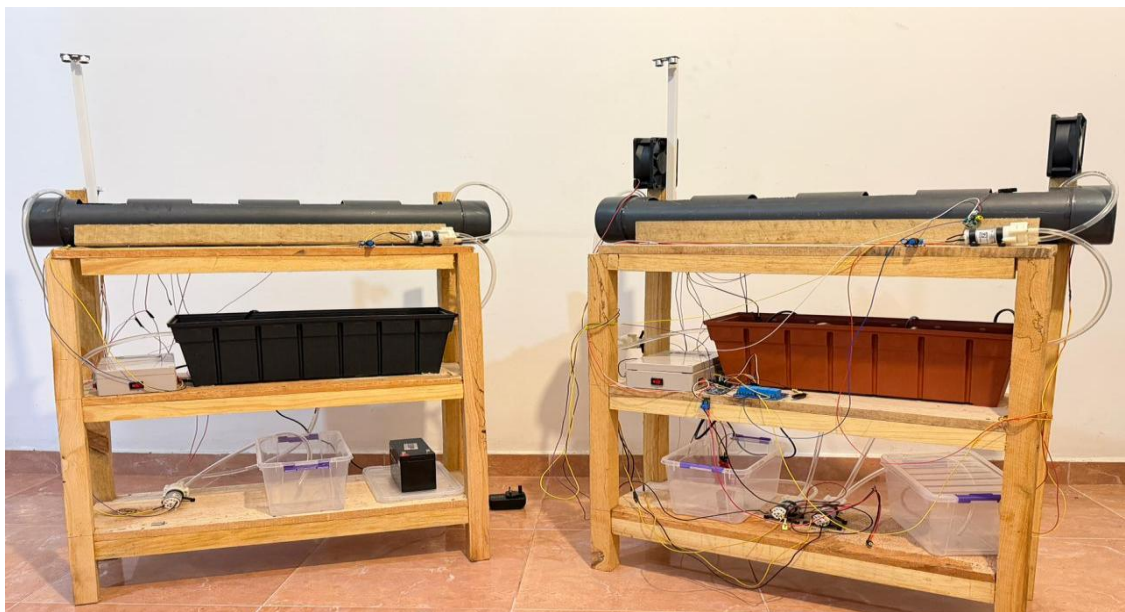
4. SUPPORTING INFORMATION

4.1 Appendices

Appendix A: Disease Detected using Mobile App



Appendix B – Inorganic and Organic Hydroponics System



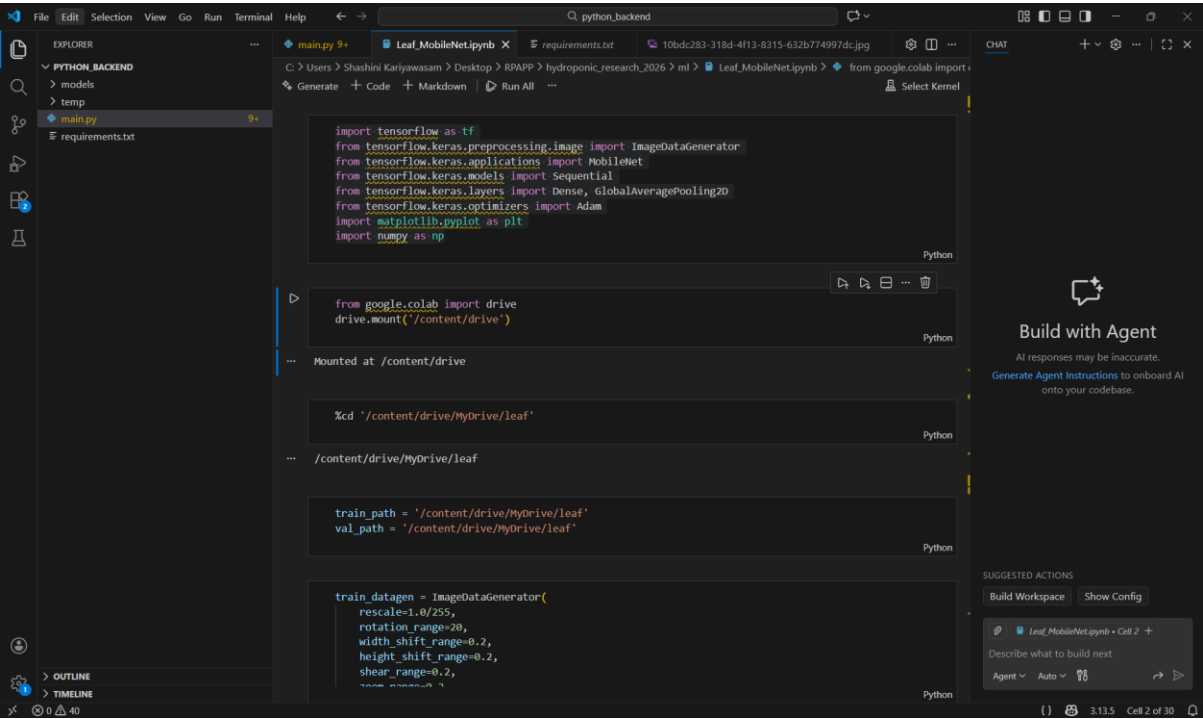
Appendix C – Hardware Components



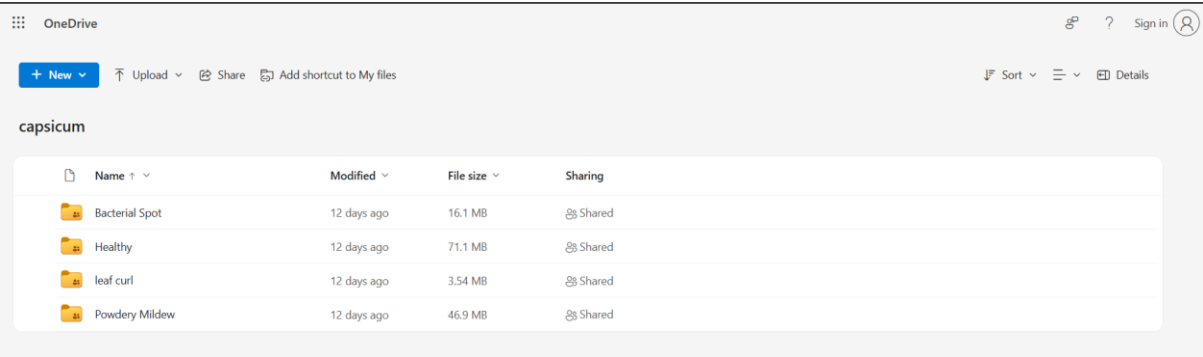
Appendix D – Flask Backend

```
File Edit Selection View Go Run Terminal Help python_backend
EXPLORER
PYTHON_BACKEND
models
leah5
temp
main.py 9+
requirements.txt
main.py > ...
1 import json, time
2 from flask import Flask, jsonify, request, make_response, send_from_directory
3 from flask_cors import CORS
4 import requests
5 import shutil
6 import cv2
7 import collections, numpy
8 import numpy as np
9 import tensorflow as tf
10 from tensorflow.keras.preprocessing.image import load_img, img_to_array
11 from keras.models import load_model
12 from tensorflow.keras.preprocessing import image
13 import tensorflow_hub as hub
14 import os
15 import pickle
16 from tensorflow.keras.utils import custom_object_scope
17 import math
18 import uuid
19 import re
20
21 leaf_model = tf.keras.models.load_model(
22     'models/leaf.h5'),
23     custom_objects={'keras_layer':hub.KerasLayer}
24 )
25
26 def leaf_find(filename):
27     classes = ['Bacterial Spot', 'Healthy', 'Powdery Mildew', 'Leaf Curl']
28     img = image.load_img(filename, target_size=(224, 224))
29     img_array = image.img_to_array(img)
30     img_processed = np.expand_dims(img_array, axis=0)
31     img_processed /= 255.
32
33     model = leaf_model
34     prediction = model.predict(img_processed)
35     prob = prediction
36
37     index = np.argmax(prediction)
38     confidence = prob[0][index]
39
40     return str(classes[index]).title(), confidence*100
41
42 UPLOAD_FOLDER = "temp/"
```

Appendix D – Machine Learning



Appendix E – Image Dataset



5. REFERENCES

- [1] R. Dhakar, B. R. Amogi, G. S. Kothawade, and L. R. Khot, "Simplified mechanistic model for estimating leaf wetness," *Agricultural and Forest Meteorology*, vol. 367, p. 110399, Mar. 2025, doi: 10.1016/j.agrformet.2025.110399.
- [2] A. Kumar and P. Savaridassan, "Monitoring and accelerating plant growth using IoT and Hydroponics," in *2023 International Conference on Computer Communication and Informatics (ICCCI)*, Coimbatore, India, Jan. 2023, pp. 1–6, doi: 10.1109/ICCCI56745.2023.10128383.
- [3] S. Skouri, S. Bouadila, R. Ayed, and S. Chehaibi, "Climate Control for Hydroponic Greenhouse: a Detailed Evaluation of Heating and Cooling Solutions," in *2025 15th International Renewable Energy Congress (IREC)*, Hammamet, Tunisia, Feb. 10.1109/IREC64614.2025.10926801.
- [4] R. Perwiratama, Y. K. Setiadi, and Suyoto, "Smart hydroponic farming with IoT based climate and nutrient manipulation system," in *2019 International Conference of Artificial Intelligence and Information Technology (ICAIIIT)*, Yogyakarta, Indonesia, 10.1109/ICAIIIT.2019.8834533. Mar. 2019, pp. 101–105, doi:
- [5] Panchal, Adesh & Patel, Subhash & Bagyalakshmi, K & Kumar, Pankaj & Khan, Ihtiram & Soni, Mukesh. (2021). Image-based Plant Diseases Detection using Deep Learning. *Materials Today: Proceedings*. 80. 10.1016/j.matpr.2021.07.281.